

Devoir surveillé

Durée 1h30

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.*

Exercice 1 (La phrase cachée - 8 pts)

Un fichier binaire contenant des données sensibles a été intercepté. Nos experts ont réussi à comprendre comme le déchiffrer et font appel à vous pour coder un programme permettant de retrouver la phrase cachée.

Une position est stockée au début du fichier (un `off_t`). Il faut se déplacer à cette position qui comprend une position gauche (un `off_t`), un caractère (un `char`) et une position droite (un `off_t`), soit au total 17 octets. Une des deux positions peut être nulle, voire les deux ; dans ce dernier cas, il s'agit d'un cul-de-sac. Pour trouver la phrase cachée, il faut explorer tous les chemins possibles. La phrase termine forcément par un point, dans un cul-de-sac.

Votre programme prend en arguments le nom du fichier à déchiffrer (le fichier d'entrée) et le nom du fichier qui contiendra la phrase cachée (le fichier de sortie).

1°) Donnez la portion de code permettant au programme d'ouvrir le fichier de sortie, tout en vérifiant qu'il n'existe pas déjà. **Vous devez gérer les erreurs dans cette question.**

2°) Nous souhaitons utiliser une pile pour parcourir le fichier. Expliquez ce qu'elle contient et comment l'utiliser.

3°) Est-il possible d'utiliser une structure pour lire les positions et le caractère en un seul appel ? Si oui, donnez la structure en C, sinon expliquez pourquoi.

4°) Comment peut-on détecter qu'une position lue est invalide ? Précisez les appels systèmes nécessaires.

5°) Donnez la portion de code permettant de lire les données situées à la position notée `pos`. Précisez les variables nécessaires éventuelles.

6°) En sachant que les caractères de la phrase trouvée sont stockés dans le fichier binaire de sortie, est-il possible de voir le contenu de ce fichier avec un éditeur de texte classique ? Expliquez.

Exercice 2 (Attrape-moi si tu peux - 7 pts)

Nous souhaitons écrire une application *ncurses* qui permet de visualiser la position de vaisseaux sur une grille de 40 par 20 (plusieurs vaisseaux peuvent se trouver sur la même case). Le programme principal est nommé `visualiseur` et les vaisseaux correspondent au programme nommé `vaisseau` qui peut être exécuté autant de fois que souhaité. Lorsque le `vaisseau` démarre, il génère une position aléatoire sur la grille puis l'envoie au `visualiseur` à l'aide d'un signal temps réel. Il se met en pause d'une seconde puis se déplace à une nouvelle position aléatoire. Il envoie cette dernière au `visualiseur`, toujours via un signal temps réel. L'utilisateur du `visualiseur` voit sur l'interface *ncurses* la position des vaisseaux représentée par un caractère 'X'. Une fenêtre d'informations affiche les créations des vaisseaux, leurs positions de départ et leurs déplacements. Le `visualiseur` s'arrête lorsque l'utilisateur presse `CRTL+C`.

1°) Comment le `visualiseur` peut-il identifier de manière unique chaque vaisseau pour n'afficher qu'un seul 'X' par vaisseau ?

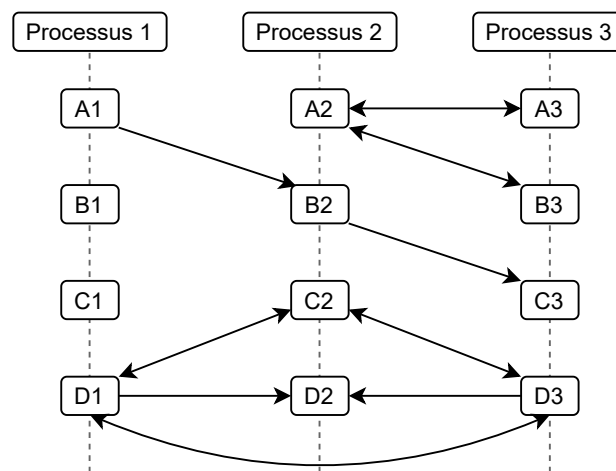
- 2°) Comment le vaisseau peut-il envoyer sa position au programme main avec un signal temps réel ? Comment peut-il le faire avec un seul signal ?
- 3°) Si le visualiseur s'arrête sans en avertir les vaisseaux, comment ces derniers peuvent-il le détecter ?
- 4°) Donnez le code permettant au serveur d'attendre un signal d'un vaisseau.

Nous supposons l'existence de la structure `position_t` qui possède 2 champs `x` et `y`, de type `unsigned char`. Nous supposons également l'existence de la fonction `deplacement` qui calcule une position aléatoire et qui la retourne via une structure `position_t`.

- 5°) Donnez le code de la boucle principale d'un vaisseau.

Exercice 3 (Sémaphores de Dijkstra et graphe d'allocation de ressources - 5 pts)

Soit le diagramme de précédence suivant, représentant 3 processus contenant chacun 4 blocs s'exécutant les uns après les autres :



- 1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).
- 2°) Proposez une solution pour réduire le nombre de sémaphores.
- 3°) Donnez un exemple de diagramme de précédence qui ne peut pas être résolu «correctement» par les sémaphores de *Dijkstra* et expliquez pourquoi.
- 4°) À l'aide d'un graphe d'allocation de ressources, montrez qu'un cycle n'induit pas forcément un interblocage. Vous expliquerez chaque élément du diagramme (en donnant un cas concret) et pourquoi il n'y a pas d'interblocage.

Annexes : quelques en-têtes de fonctions C

```

void          _exit(int code);
unsigned int  alarm(unsigned int nb_secondes);
int           atexit(void (*procedure)(void));
void          chucknorris(int god);
void          clearerr(FILE *flux);
int           close(int fd);
int           closedir(DIR *repertoire);
int           creat(const char *chemin, mode_t mode);
int           dup(int ancien_fd);
int           dup2(int ancien_fd, nouveau_fd);
int           execve(const char *fichier, char *const argv[], char *const envp[]);
void          exit(int statut);
int           fcntl(int fd, int commande, ...);
int           fclose(FILE *flux);
FILE          *fdopen(int fd, const char *mode);
int           feof(FILE *flux);
int           ferror(FILE *flux);
int           fflush(FILE *flux);
int           fileno(FILE *flux);
FILE          *fopen(const char *chemin, const char *mode);
pid_t         fork();
size_t        fread(void *tampon, size_t taille, size_t nb, FILE *flux);
FILE          *freopen(const char *chemin, const char *mode, FILE *flux);
int           fseek(FILE *flux, long offset, int point_depart);
int           fstat(int fd, struct stat *tampon);
int           ftell(FILE *flux);
size_t        fwrite(const void *tampon, size_t taille, size_t nb, FILE *flux);
pid_t         getpid();
pid_t         getppid();
int           kill(pid_t pid, int numero_signal);
off_t         lseek(int fd, off_t offset, int point_depart);
int           lstat(const char *chemin, struct stat *tampon);
void          *memcpy(void *destination, const void *source, size_t taille);
void          *memset(void *tampon, int caractere, size_t nombre_elements);
int           mkfifo(const char *nomFichier, mode_t mode);
int           open(const char *chemin, int options);
int           open(const char *chemin, int options, mode_t mode);
DIR           *opendir(const char *chemin);
int           pause(void);
int           pipe(int tube[2]);
ssize_t       read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void          rewind(FILE *flux);
void          rewinddir(DIR *repertoire);
void          (*signal(int numero_signal, void (*handler)(int)))(int)
int           sigaction(int numero_signal, const struct sigaction *action,
                        struct sigaction *ancienne_action);
int           sigaddset(sigset_t *ensemble, int numero_signal);
int           sigdelset(sigset_t *ensemble, int numero_signal);
int           sigemptyset(sigset_t *ensemble);
int           sigfillset(sigset_t *ensemble);
int           sigismember(sigset_t *ensemble, int numero_signal);
int           sigpending(sigset_t *ensemble);
int           sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int           sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int           sigsuspend(const sigset_t *ensemble);
int           sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int           sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                           const struct timespec *timeout);
unsigned int  sleep(unsigned int nombre_secondes);

```

```

int          stat(const char *chemin, struct stat *tampon);
time_t       time(time_t *temps);
int          truncate(const char *chemin, off_t offset);
int          ftruncate(int fd, off_t offset);
int          truncate64(const char *chemin, off64_t offset);
int          ftruncate64(int fd, off64_t offset);
int          unlink(const char *nomFichier);
pid_t        wait(int *statut);
pid_t        waitpid(pid_t pid, int *statut, int options);
ssize_t      write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_départ (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED
valeurs de errno : E2BIG, EACCESS, EAGAIN, EBADE, EBADE, EBADF, EBADFD, EBADMSG, EBADR, EBADRQC, EBADSLT, EBUSY, ECANCELED, ECHILD, ECHRNG, EDEADLK, EDOM, EDQUOT, EEXIST, EFAULT, EFBIG, EHOSTDOWN, EIDRM, EILSEQ, EINPROGRESS, EINTR, EINVAL, EIO, EISDIR, EISNAM, ELOOP, EMEDIUMTYPE, EMFILE, EMLINK, EMSGSIZE, EMULTIHOP, ENAMETOOLONG, ENFILE, ENOBUFS, ENODATA, ENODEV, ENOENT, ENOEXEC, ENOKEY, ENOLCK, ENOLINK, ENOMEDIUM, ENOMEM, ENOMSG, ENOSPC, ENOSR, ENOSTR, ENOSYS, ENOTBLK, ENOTCONN, ENOTDIR, ENOTEMPTY, ENOTSUP, ENOTTY, EOVERFLOW, EPERM, EPFNOSUPPORT, EPIPE, ERANGE, ERESTART, EROFS, ESPIPE, ESRCH, ESTALE, ESTRPIPE, ETIME, ETXTBSY, EUCLEAN, EXDEV, EXFULL, ECHUCKTOOSTRONG

Annexes : macros

Sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG
 Sur le champ **st_mode** de struct stat : S_ISREG, S_ISDIR, S_ISFIFO

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};

union sigval_t {
    int sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```